

Data Structure

09. 힙과 DP

9주차: Heap, Priority Queue, DP

? 오늘 배울 거 요약

↓ 1부: 힙과 우선순위 큐

- Queue 복습 & Priority Queue
- Complete Binary Tree
- Min Heap / Max Heap의 구조 및 특징
- Heap 삽입/삭제 (Sift Up / Sift Down)

🔍 2부: Dynamic Programming

- 단순 재귀의 문제점
- Memoization & Tabulation
- DP의 2가지 조건
- DP 문제 접근 절차 (4단계)

목차 INDEX

01 Priority Queue

- Queue
- Priority Queue
- 구현 방식
- 시간복잡도

02 Heap

- Complete Binary Tree
- Heap
- Min/Max Heap
- Sift Up/Down
- Heapify
- Heap Sort

03 Dynamic Programming

- Dynamic Programming
- Overlapping Subproblems
- Memoization
- Tabulation
- DP의 4단계 절차
- 1D/2D DP

01

우선순위 큐

Priority Queue

Queue (7주차)

- 선입선출 (FIFO) 구조
- 큐로 데이터가 들어오는걸 Enqueue, 나가는걸 Dequeue라 함

01 큐 Queue

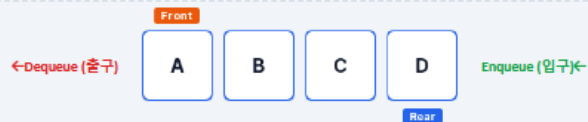
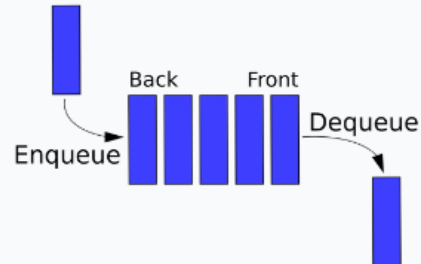
Enqueue & Dequeue

Queue (큐)

"먼저 들어온 데이터가 먼저 나오는 FIFO 선형 자료구조"

큐로 들어오는걸 Enqueue,
큐에서 나가는걸 Dequeue라 함

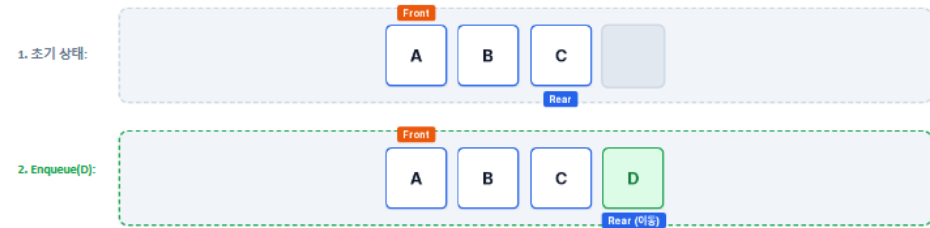
FIFO : First In, First Out (선입선출)



01 큐 Queue

Enqueue

Enqueue는 데이터가 Queue의 Rear(끝)쪽으로 들어오는 연산



✓Enqueue가 수행되면 Rear 포인터가 오른쪽으로 1칸 이동하면서 새 데이터를 가리킴

01 큐 Queue

Dequeue

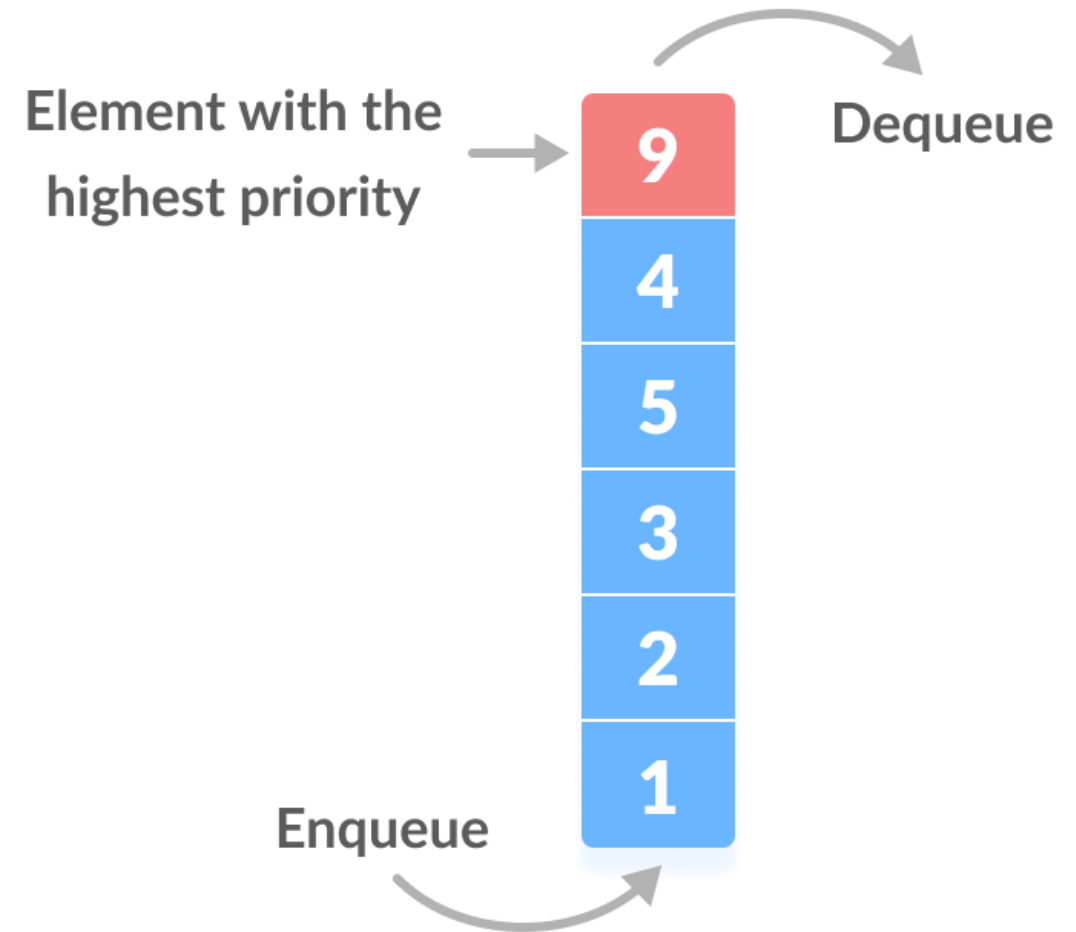
Dequeue는 Queue에 가장 먼저 들어온 Front(시작)쪽 데이터를 제거하는 연산



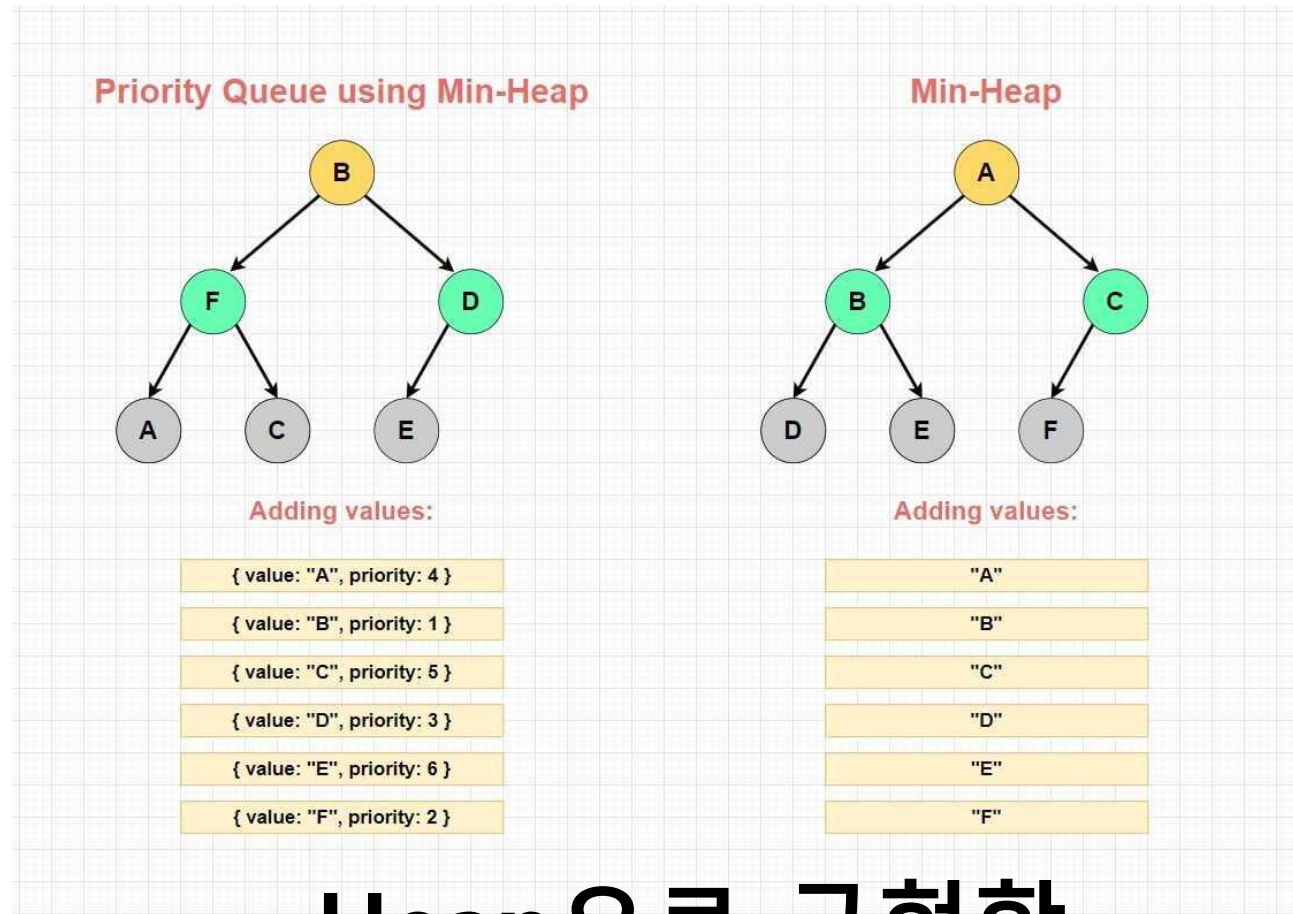
✓Dequeue가 수행되면 A가 제거되면서 Front 포인터가 다음 요소(B) 위치로 1칸 이동함

Priority Queue

- 우선순위가 높은 데이터부터 처리하는 구조
- 들어온 순서와 관계없이 우선순위에 따라 처리 순서 결정
- 데이터 삽입은 insert, 가장 우선순위가 높은 데이터 제거는 pop이라 함 (Enqueue/Dequeue 로도 표현함)
- 데이터마다 우선순위라는 Key를 부여해서 처리 시 가장 높은 Key를 우선 선택하는 방식
- OS의 프로세스 스케줄링, Dijkstra Algorithm 등에서 쓰임



Priority Queue의 구현



Heap으로 구현함

(정렬되지 않은 배열이나 정렬된 배열으로도 구현 자체는 가능)

Priority Queue의 구현에 Heap을 쓰는 이유

구현 방식	삽입 (Insert)	최우선 요소 삭제 (Pop)	비고 및 특징
정렬되지 않은 배열	$O(1)$ 맨 뒤에 추가	$O(N)$ 전체 탐색 필요	삭제 시 매번 최댓값/최솟값을 찾아야 함
정렬된 배열	$O(N)$ 정렬 위치 찾기+시프트	$O(1)$ 맨 끝 데이터 pop	삽입 시 기존 원소들을 밀어내야 함
힙 (Heap)	$O(\log N)$	$O(\log N)$	삽입과 삭제 모두 균형있게 빠르게 처리

Heap으로 구현하면 데이터가 100만개 있어도 약 20번의 연산으로 값 찾기 가능

02

합

Heap

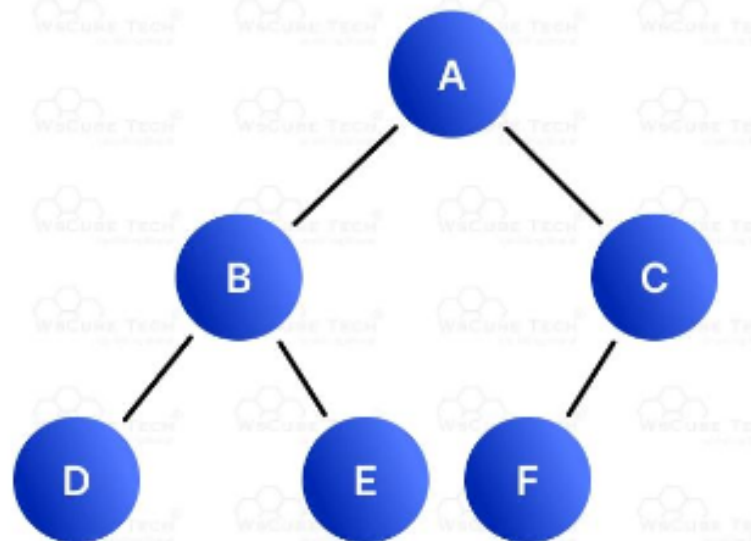
Complete Binary Tree (8주차)

01 트리 Tree

완전 이진 트리 (Complete Binary Tree)

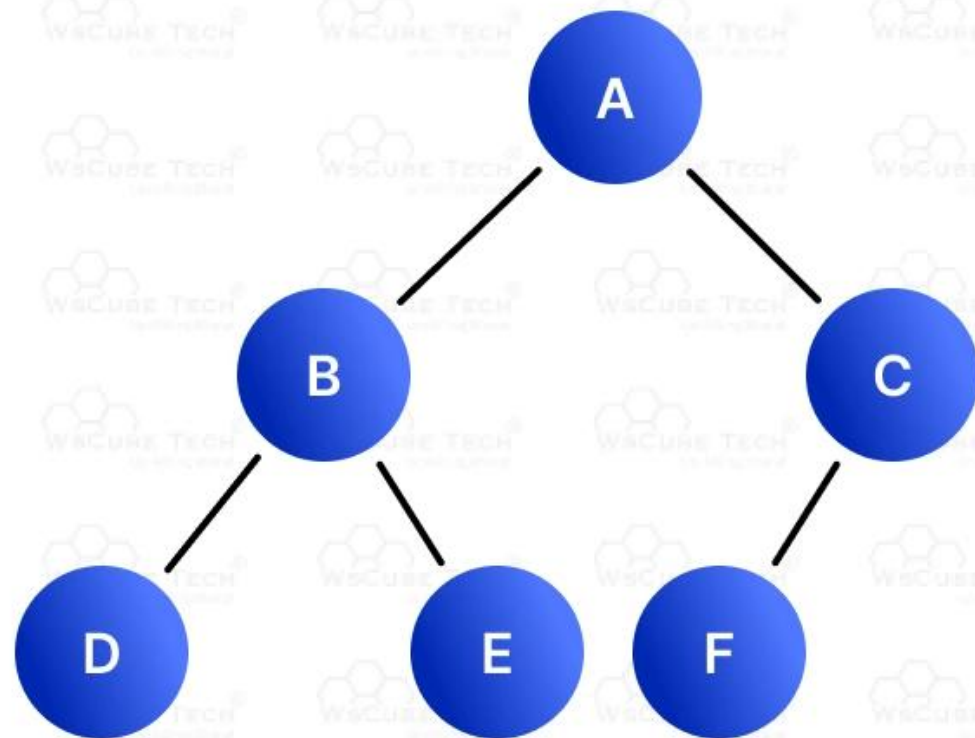
1. 마지막 레벨을 제외한 모든 레벨은 노드가 가득 차 있어야 함
(왼쪽, 오른쪽 모두 노드가 있어야 함)
2. 마지막 레벨의 노드들은 왼쪽부터 차례대로 채워져야 함

- Heap(9주차) 의 기반



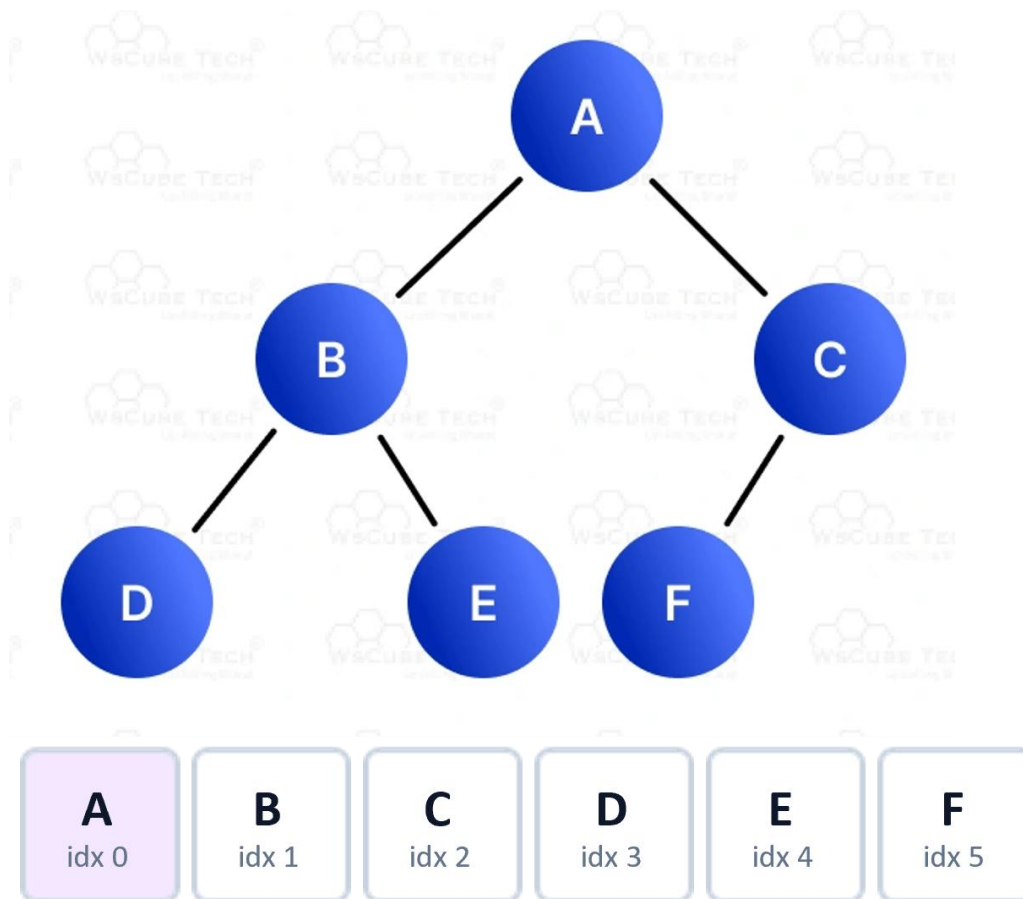
Complete Binary Tree (8주차)

- Child가 2개 이하인 이진 트리
- 마지막 레벨을 제외한 모든 레벨이 노드로 꽉 채워져 있음
- 마지막 레벨의 노드들은 왼쪽부터 차례대로 채워짐
- 중간에 빈 공간이 존재할 수 없음



Complete Binary Tree를 배열로 매핑하기

- 완전 이진 트리는 그 특성상 각 노드의 주소를 저장할 필요가 없음
(Parent/Child의 위치를 계산 가능)
- 이를 배열로 옮기면 배열의 인덱스만으로 각 노드의 Parent/Child의 위치를 계산 가능
- 배열에 저장할 땐 위에서 아래로, 왼쪽에서 오른쪽으로 저장



인덱스로 부모/자식 노드 계산

부모 노드 (Parent)

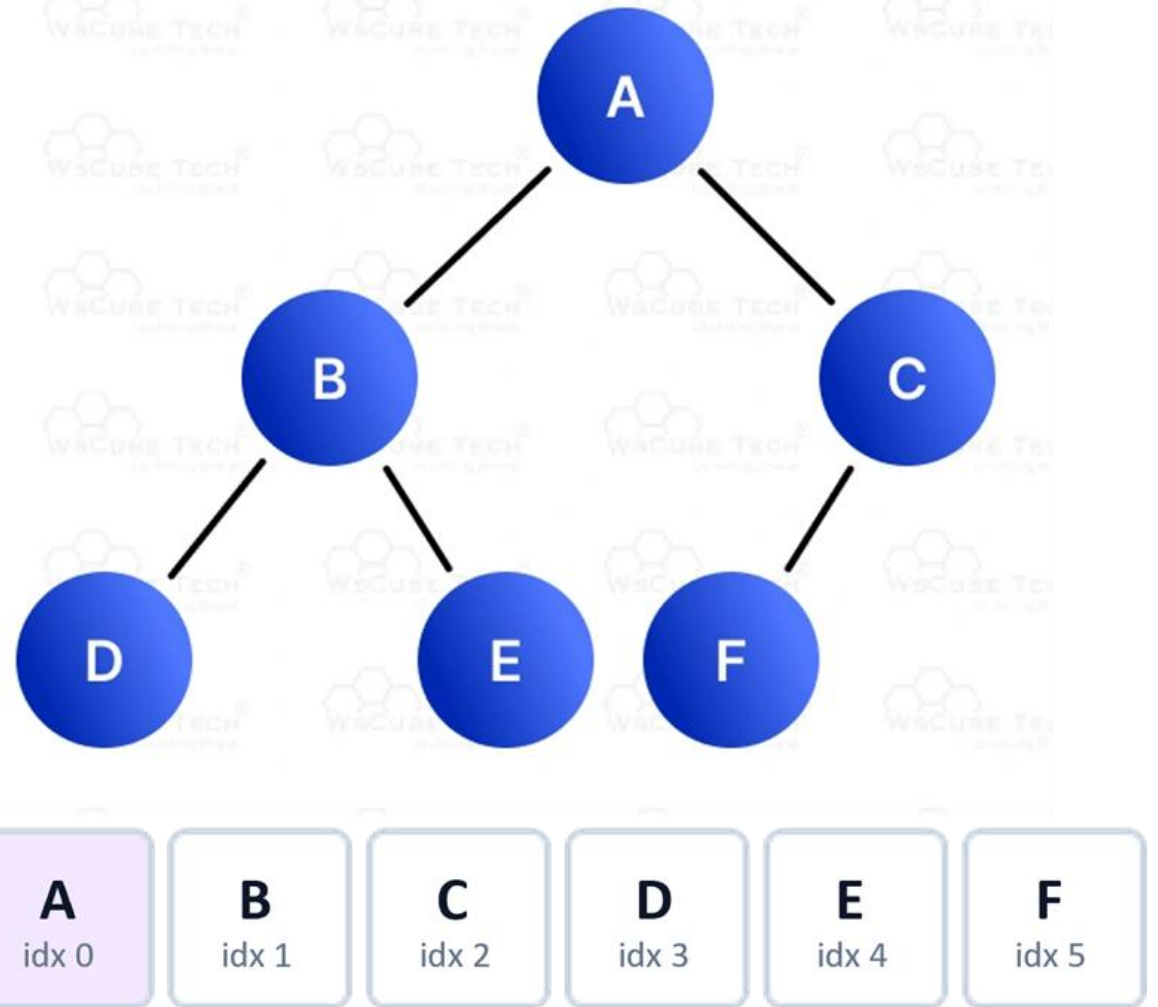
$$(i - 1) // 2$$

Left Child

$$2 * i + 1$$

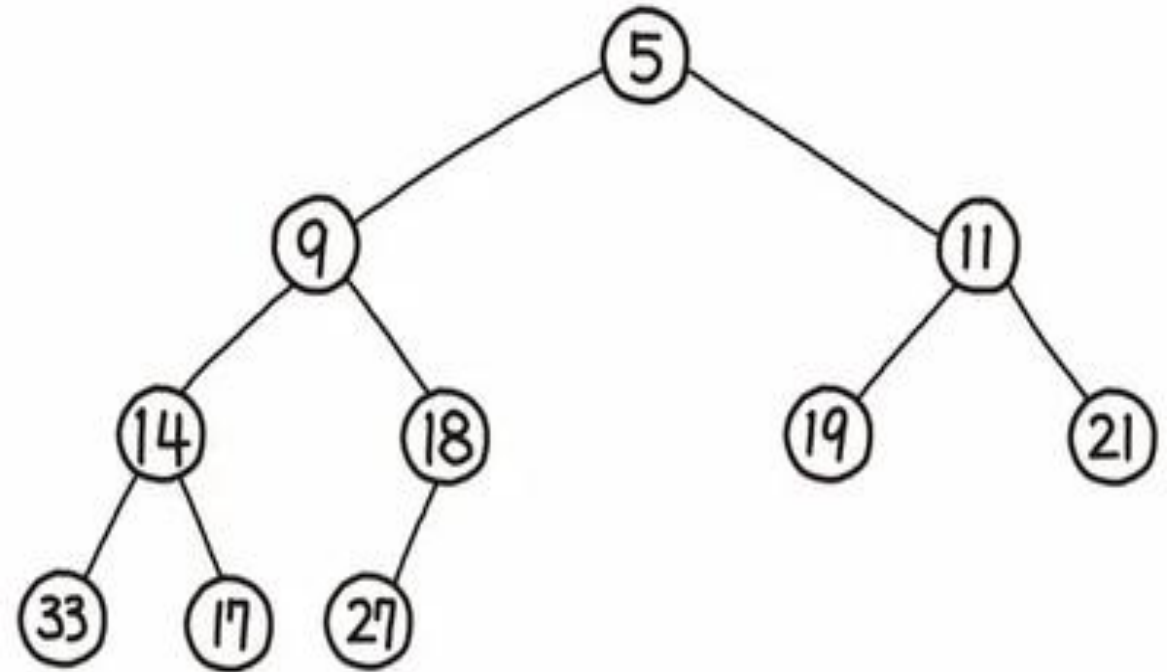
Right Child

$$2 * i + 2$$

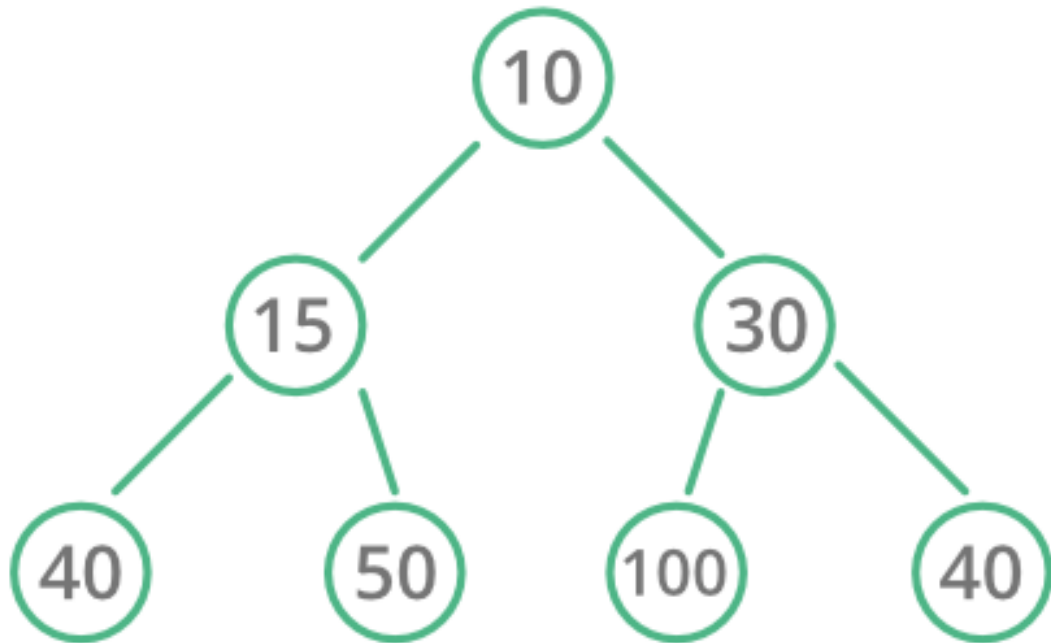


Heap Tree (or Heap)

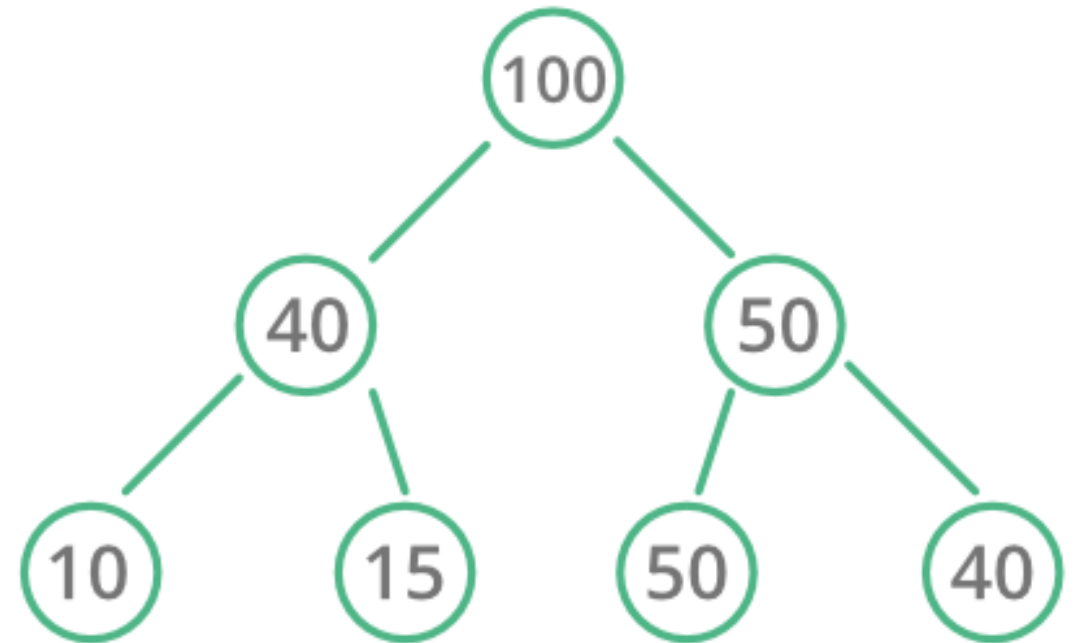
- 완전 이진 트리의 조건을 만족해야 함
- 부모와 자식 노드 사이 크기 규칙을 만족해야 함
(이 규칙에 따라 무슨 힙인지가 갈림)
- 부모가 항상 자식보다 크면 Max Heap,
항상 작으면 Min Heap이라 부름
- 오른쪽 이미지는 Min Heap을 나타냄
- Heap의 루트 노드는 항상 최댓값 또는
최솟값이 있기 때문에 최솟값 최댓값
확인 연산(Peek)이 $O(1)$ 임



Min Heap & Max Heap

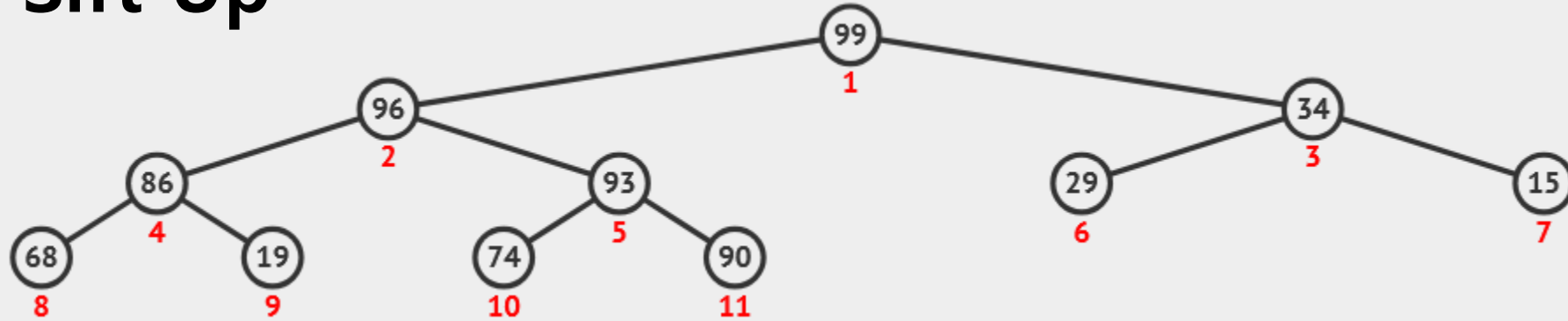


Min Heap

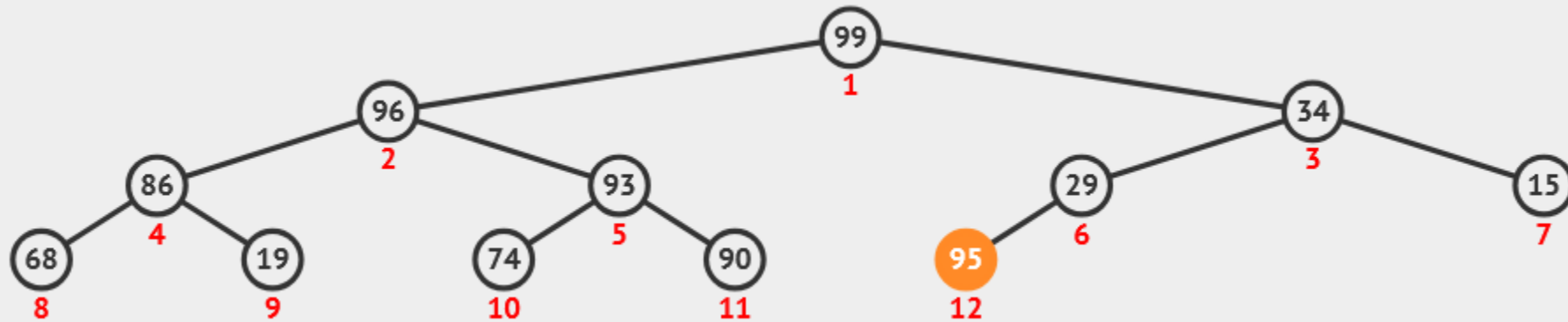


Max Heap

Insert: Sift Up

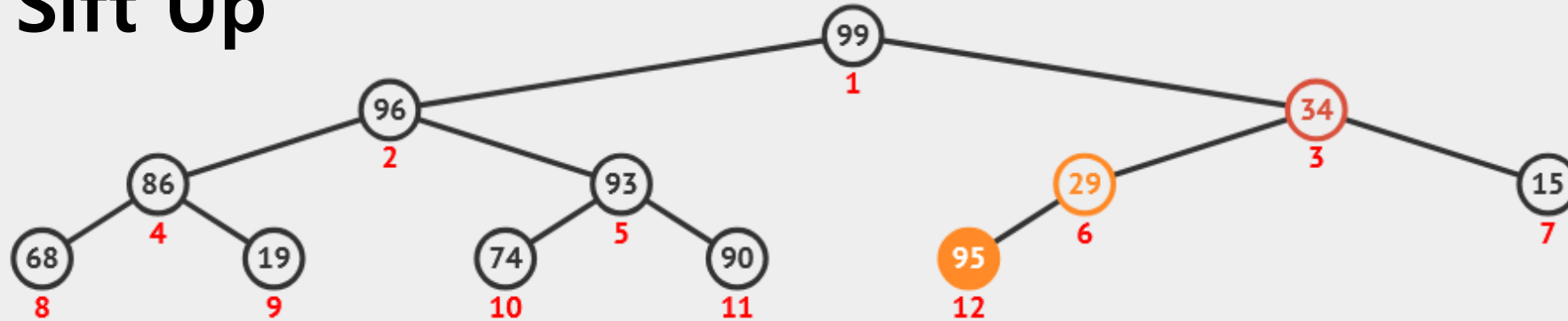


1. 간단한 Max Heap에 95라는 값을 넣고 싶은 상황

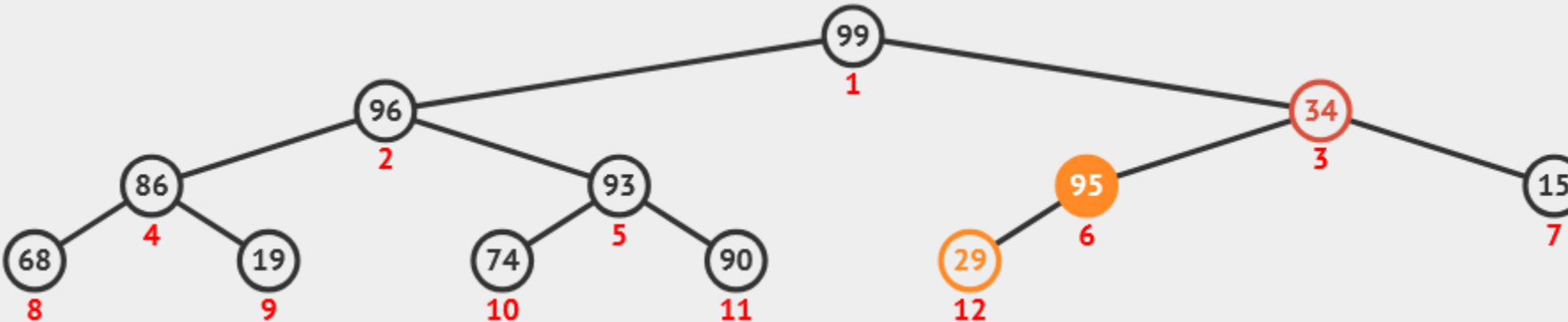


2. 완전 이진 트리의 규칙을 만족해야 하므로 빈 공간에 노드 생성

Insert: Sift Up



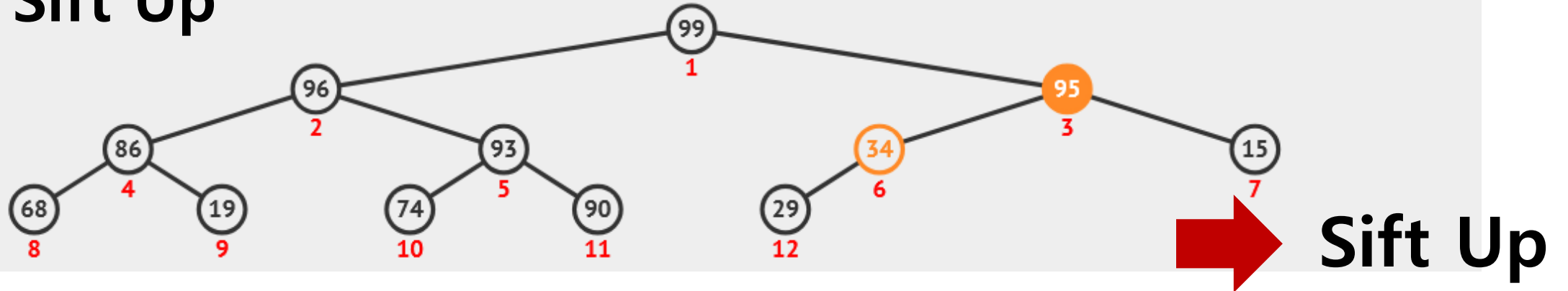
3. Max Heap의 크기 규칙 (부모는 항상 자식보다 커야 함) 검증
검증 결과 부모 중 29, 34가 95보다 작음



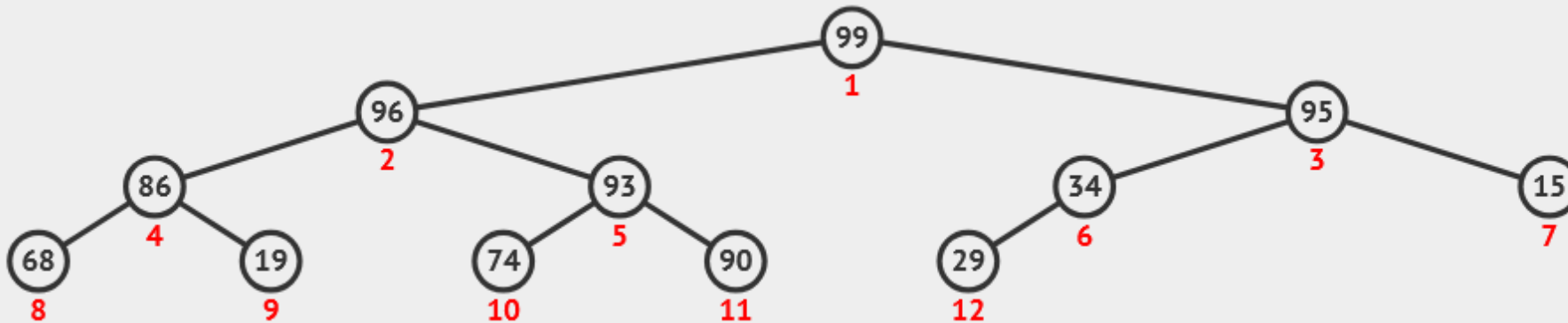
4. 일단 29와 위치를 바꿔줌.
새로운 부모인 34가 여전히 95보다 작음

 **Sift Up**

Insert: Sift Up



5. Max Heap의 크기 규칙을 만족시키기 위해 95를 한 레벨 위로 올려줌



6. 삽입 연산이 완료된 Max Heap의 모습

(Insert 연산 중 Sift Up 연산을 수행했고, 그 과정에서 2번의 Swap 발생 이라고 표현하면 됨)

Insert 연산의 시간복잡도

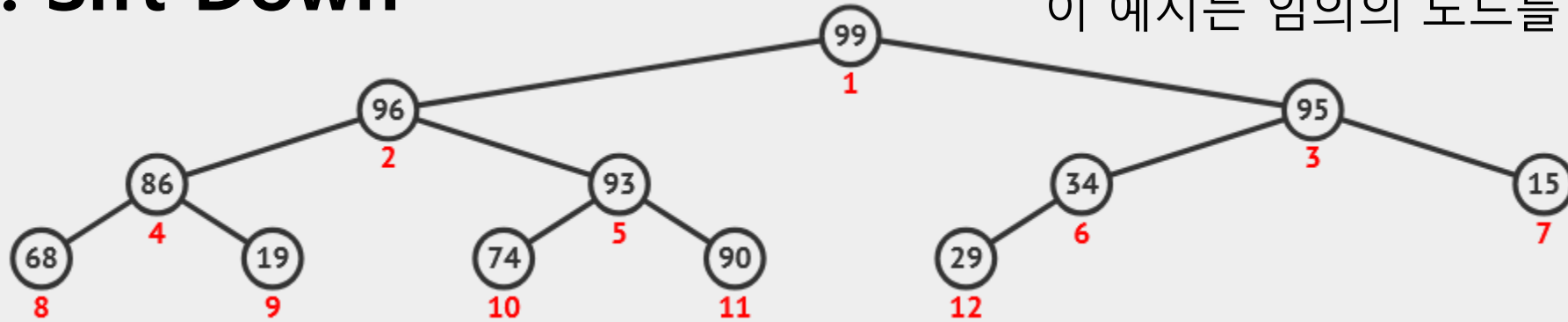
구현 방식	삽입 (Insert)	최우선 요소 삭제 (Pop)	비고 및 특징
정렬되지 않은 배열	$O(1)$ 맨 뒤에 추가	$O(N)$ 전체 탐색 필요	삭제 시 매번 최댓값/최솟값을 찾아야 함
정렬된 배열	$O(N)$ 정렬 위치 찾기+시프트	$O(1)$ 맨 끝 데이터 pop	삽입 시 기존 원소들을 밀어내야 함
힙 (Heap)	$O(\log N)$	$O(\log N)$	삽입과 삭제 모두 균형있게 빠르게 처리

최악의 경우(추가하는 노드가 맨 밑에서 시작해 Root까지 가야 하는 경우)의
복잡도는 $O(\log N)$ 임

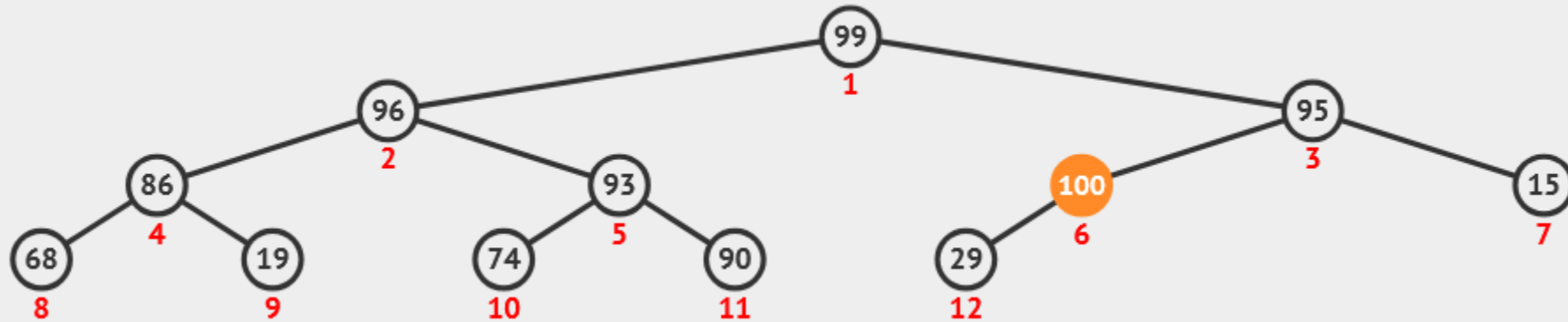
$N = 1,000,000$ 일 때 약 20번 이동해야 함
(정렬된 배열은 최악의 경우 백만번 이동)

Delete: Sift Down

일반적인 Delete는 그냥 Root를 삭제함
이 예시는 임의의 노드를 삭제하는 방법임

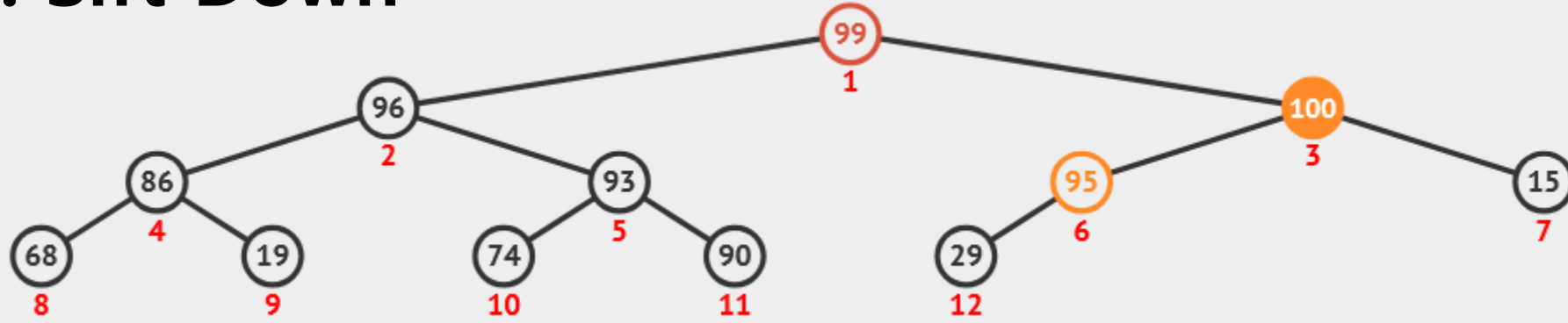


1. 힙의 삭제는 항상 Root를 삭제하는 식으로 이루어짐.
34라는 값(6번 인덱스)를 삭제하고 싶은 상황

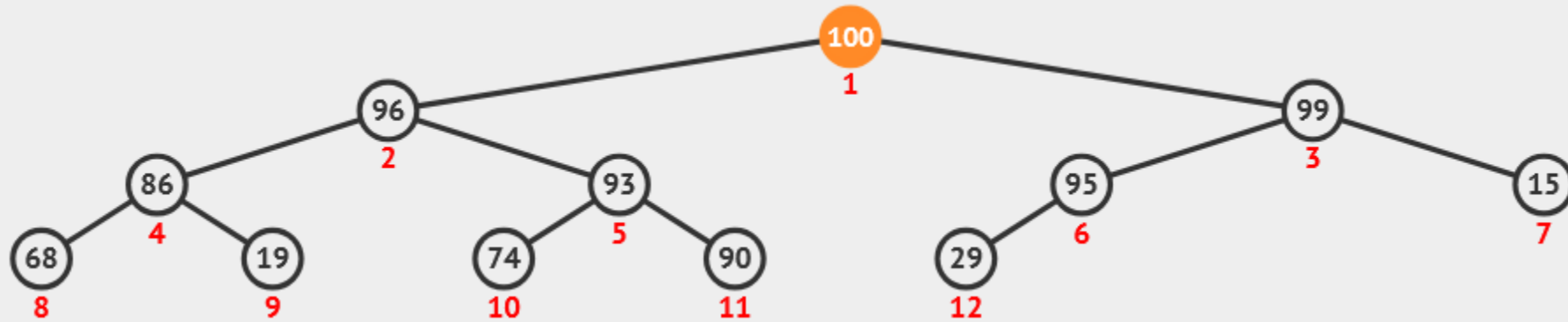


2. 힙의 삭제는 항상 Root를 삭제하므로 해당 인덱스의 원소를
최댓값(기존 Root) + 1로 설정해줌

Delete: Sift Down

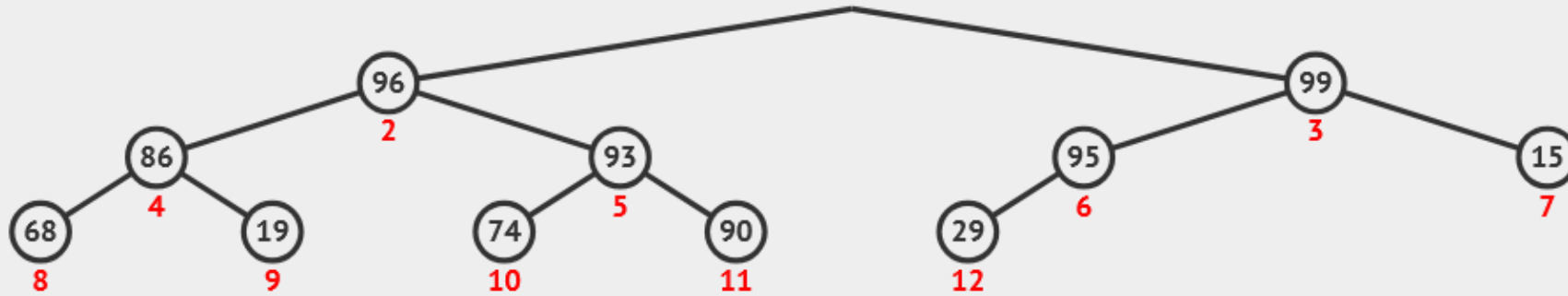


3. Max Heap의 규칙을 만족시키기 위해 Sift Up 수행

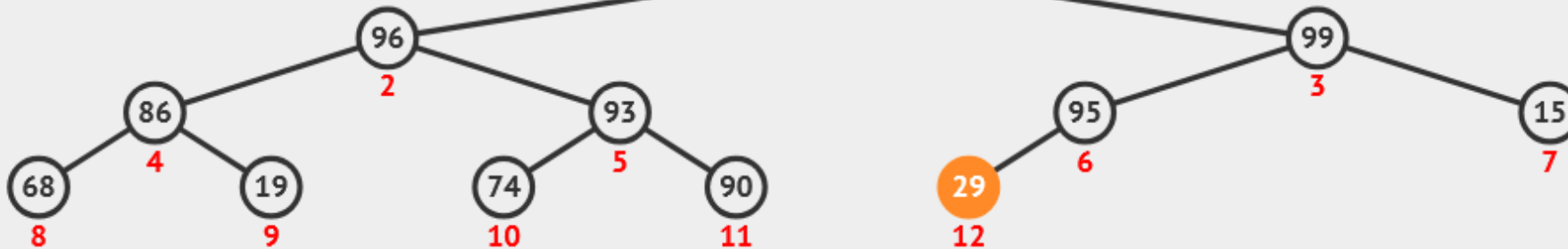


4. 한번 더 수행

Delete: Sift Down



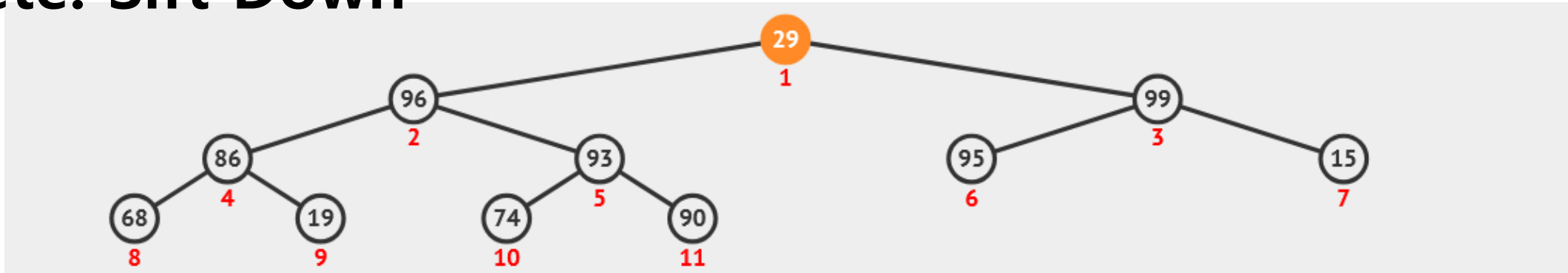
5. Root 제거



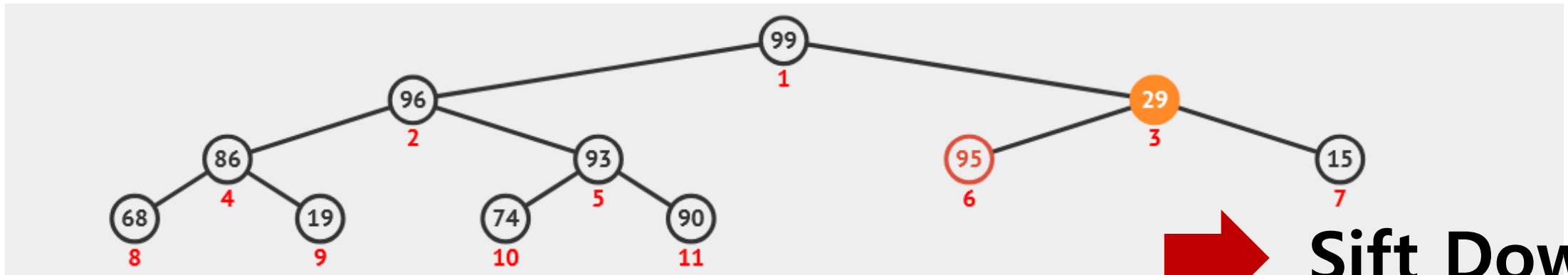
6. Root에 새로운 노드를 넣어야 함.

항상 완전이진트리의 형태를 유지해야 하므로 마지막 인덱스 노드를 Root로 넣으면 됨

Delete: Sift Down



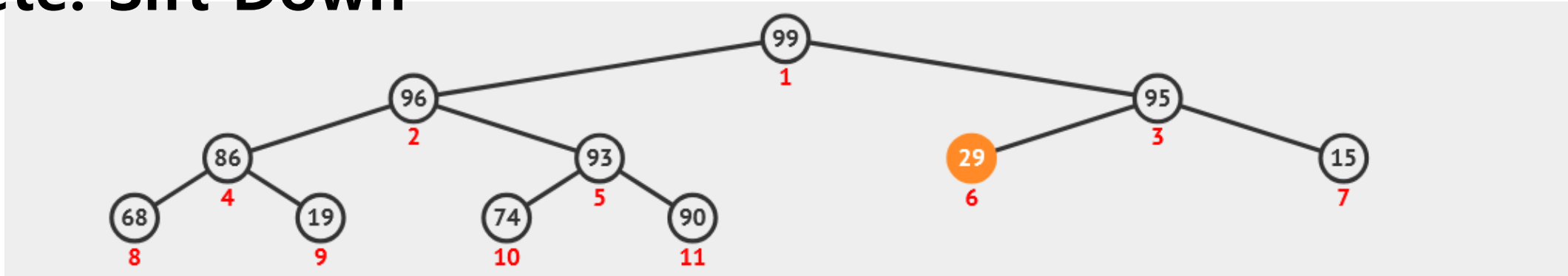
7. 가장 마지막 원소를 Root로 올린 모습



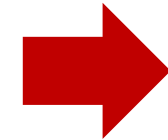
Sift Down

8. Max Heap의 크기 규칙을 만족시키기 위해 자식 중 가장 큰 원소와 Swap 수행함

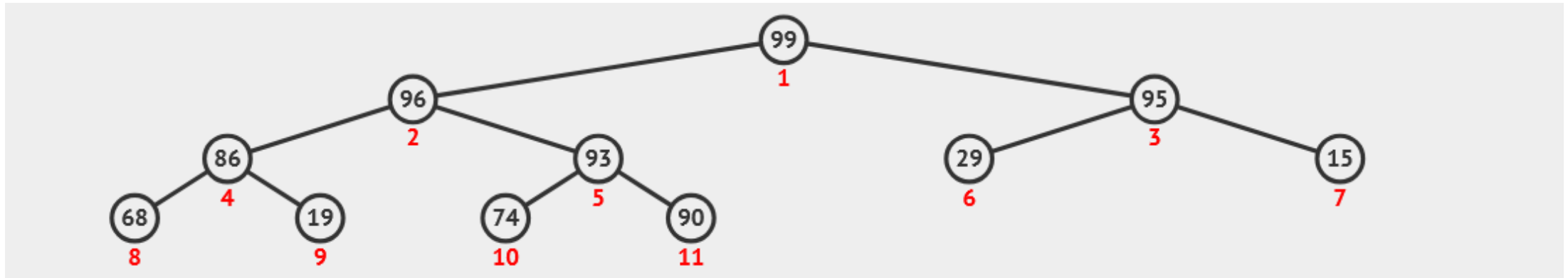
Delete: Sift Down



9. 한번 더 자식 중 가장 큰 값과 Swap함



Sift Down



10. Max Heap에서 임의의 원소 제거하기 성공

Delete 연산의 시간복잡도

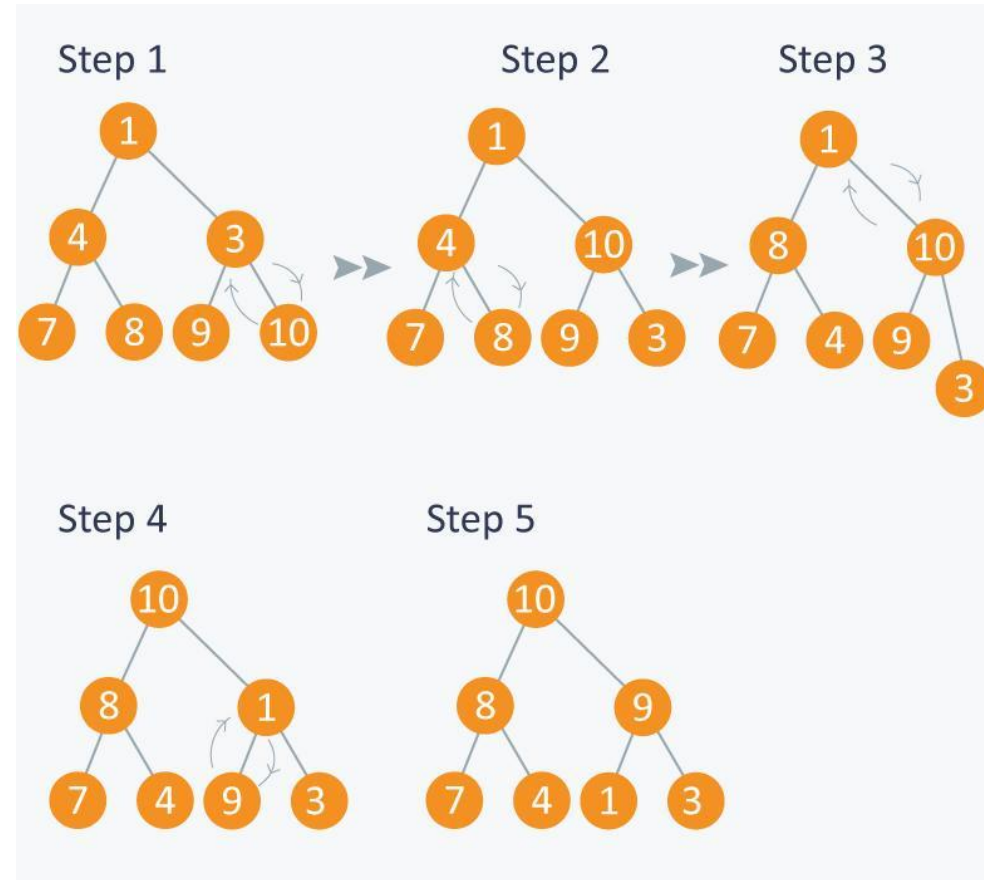
구현 방식	삽입 (Insert)	최우선 요소 삭제 (Pop)	비고 및 특징
정렬되지 않은 배열	$O(1)$ 맨 뒤에 추가	$O(N)$ 전체 탐색 필요	삭제 시 매번 최댓값/최솟값을 찾아야 함
정렬된 배열	$O(N)$ 정렬 위치 찾기+시프트	$O(1)$ 맨 끝 데이터 pop	삽입 시 기존 원소들을 밀어내야 함
힙 (Heap)	$O(\log N)$	$O(\log N)$	삽입과 삭제 모두 균형있게 빠르게 처리

최악의 경우(노드(Root)를 삭제한 후 마지막 노드를 Root로 이동한 다음 Sift Down을 마지막 레벨까지 하는 경우)의 복잡도는 $O(\log N)$ 임

$N = 1,000,000$ 일 때 약 20번 이동해야 함 (이진트리라 $\log_2(1,000,000) \approx 20$)
(정렬된 배열은 최악의 경우 백만번 이동)

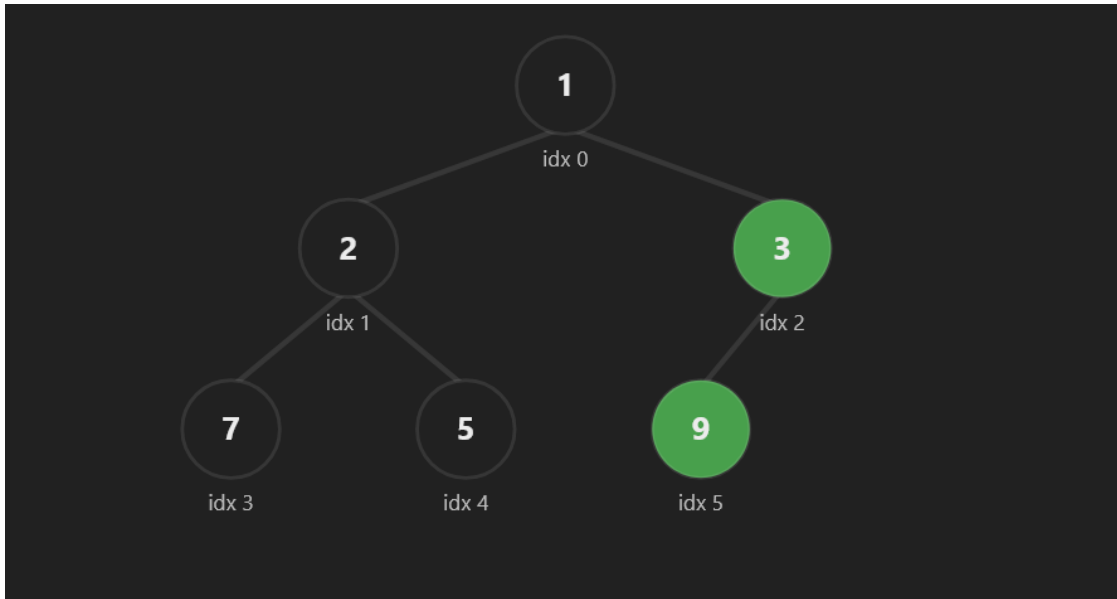
Heapify

- 힙의 조건을 만족하도록 트리(또는 배열)를 재정렬하는 과정
- 특정 노드를 기준으로 부모 노드와 자식 노드의 값을 비교하여 올바른 위치로 이동시킴
- 힙의 조건을 만족하지 않는 경우 부모와 자식의 위치를 교환(Swap)
- 교환한 이후에도 조건을 만족하지 않으면 아래 방향으로 비교와 교환을 반복
- Heap에서 데이터를 삽입하거나 삭제한 후 힙 구조를 복구할 때 사용됨

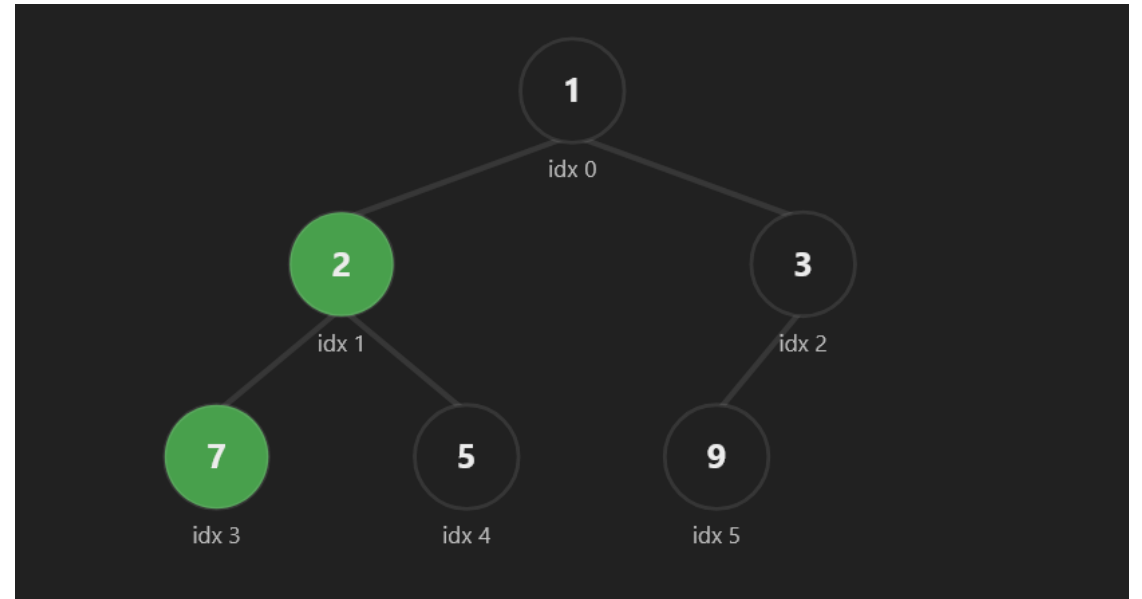


Heapify

- 힙으로 만들 배열: [7, 2, 9, 1, 5, 3]



Push + Sift Up
 $O(N \log N)$



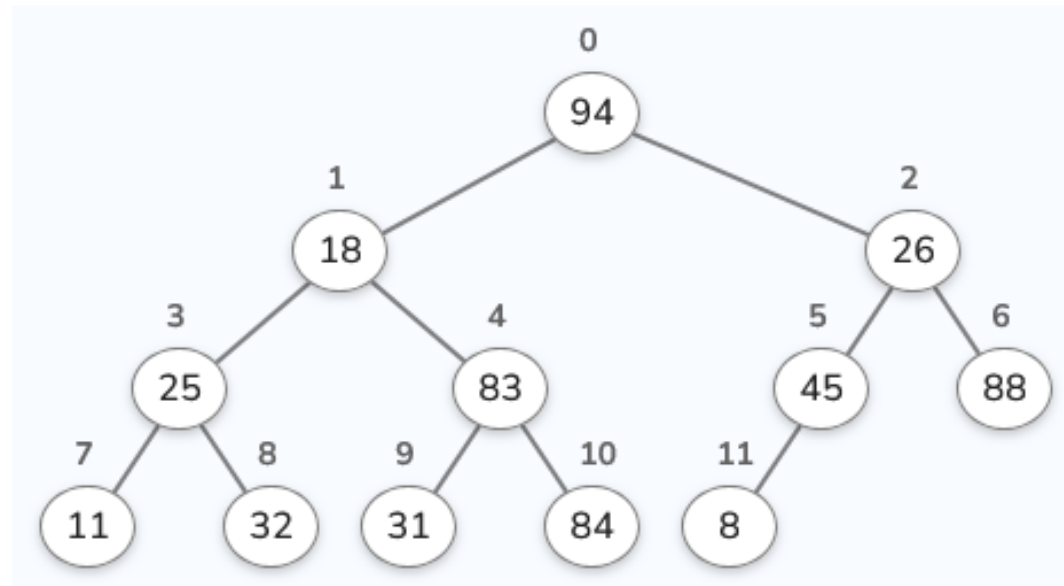
Bottom-up
 $O(N)$

Heap Sort

1. 정렬되지 않은 배열:
[94, 18, 26, 25, 83, 45, 88, 11, 32, 31, 84, 8]



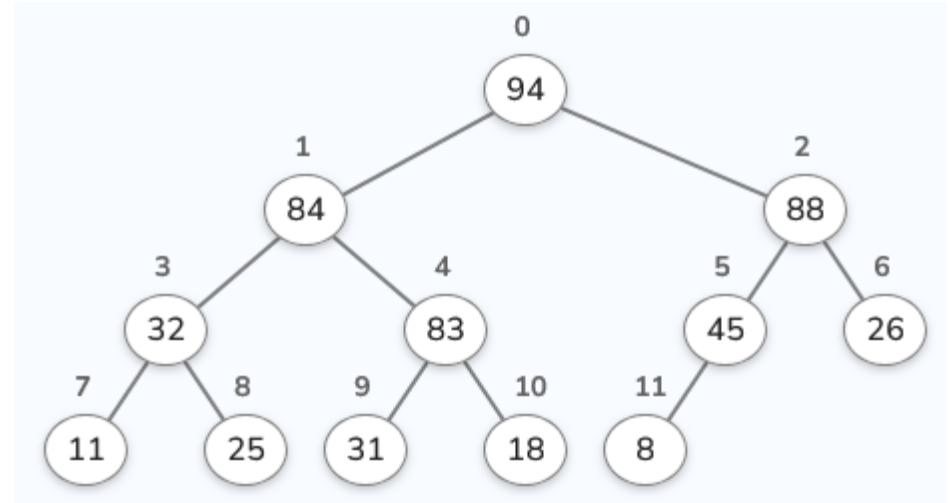
2. 배열을 완전 이진 트리로 만들
(아직 Heapify는 안함)



힙의 Root에는 항상 최솟값/최댓값이 온다는 사실을 이용

Heap Sort

3. Heapify 된 완전 이진 트리
(Max Heap이 되게끔 Heapify)

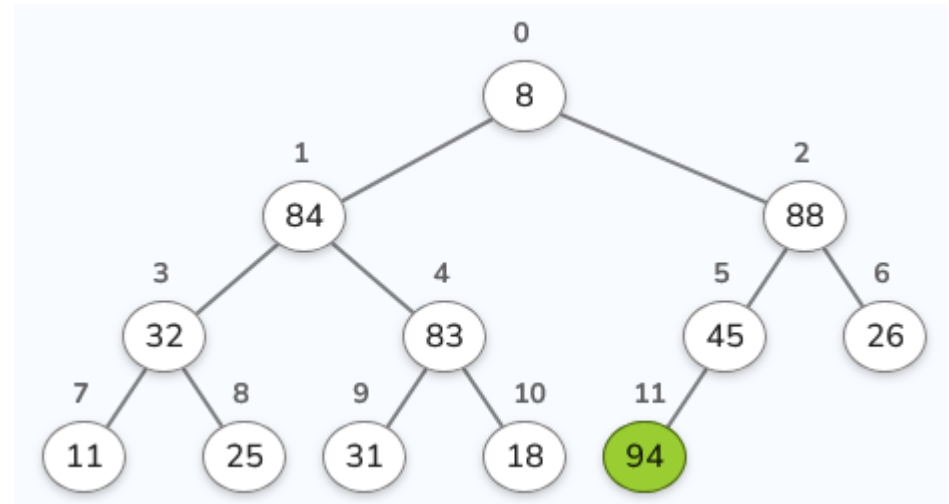


4. Max Heap으로 Heapify된 Heap의 Root는 항상 최댓값임
이 최댓값을 기존 배열의 맨 뒤에 append 하고,
남은 원소들에서 Sift Down을 수행함

현재 배열: [18, 26, 25, 83, 45, 88, 11, 32, 31, 84, 8 | 94]

Heap 영역

정렬 완료 영역



힙의 Root에는 항상 최솟값/최댓값이 온다는 사실을 이용

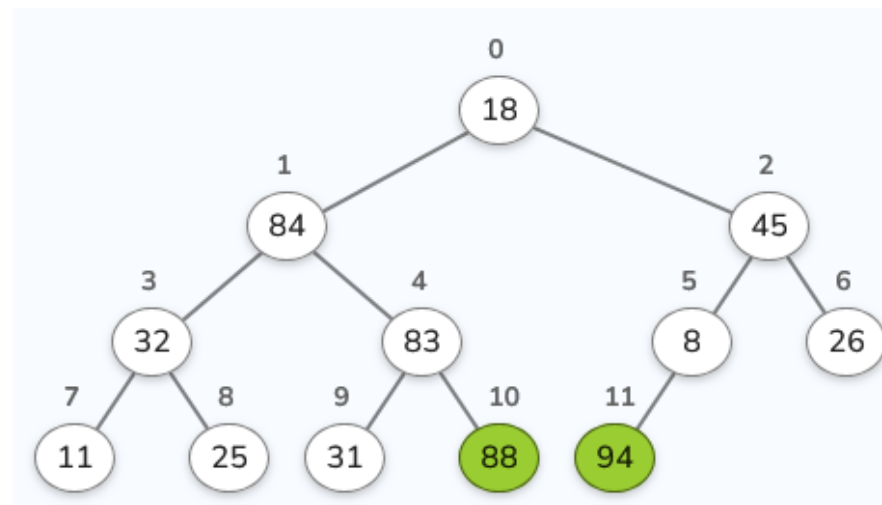
Heap Sort

5. 같은 방법을 반복해서 다음 최댓값인 88이 Root로 올라감
그 Root 값을 처음 append 한 94 앞으로 보냄

현재 배열: [18, 26, 25, 83, 45, 11, 32, 31, 84, 8 | 88, 94]

Heap 영역

정렬 완료 영역

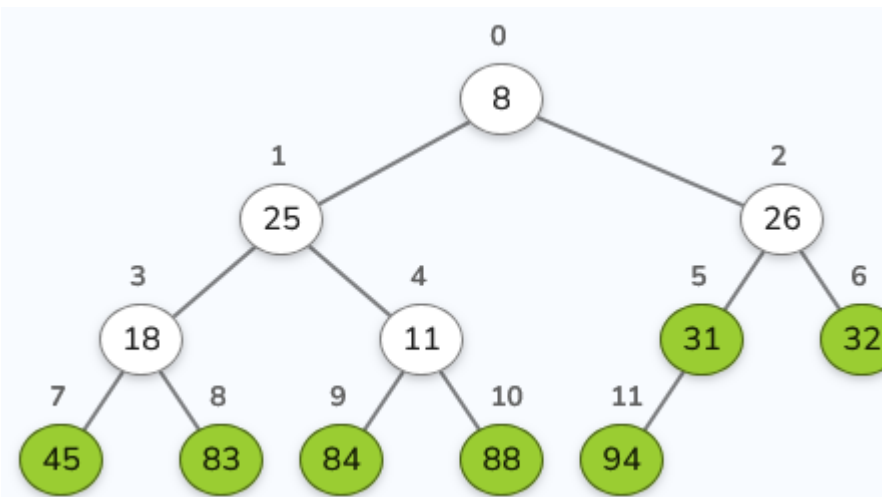


6. 모든 원소가 정렬될 때까지 위의 과정을 반복함

현재 배열: [18, 26, 25, 11, 8 | 31, 32, 45, 83, 84, 88, 94]

Heap 영역

정렬 완료 영역

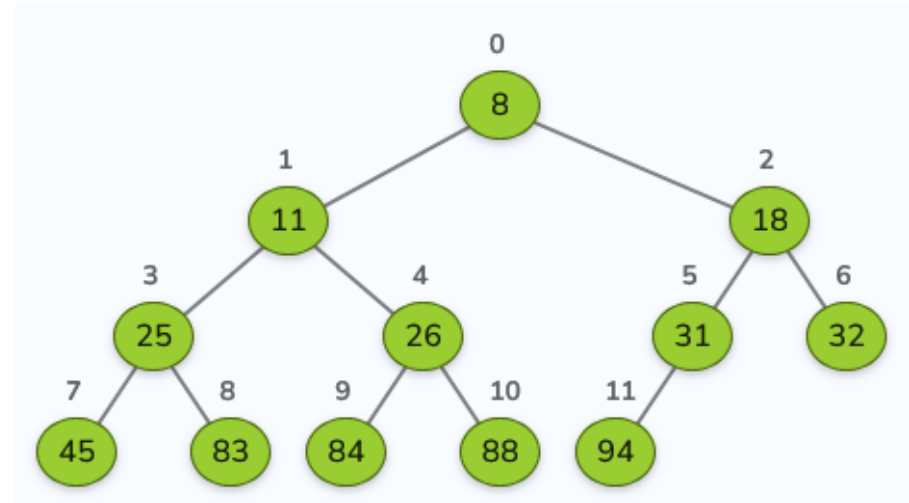


힙의 Root에는 항상 최솟값/최댓값이 온다는 사실을 이용

Heap Sort

7. 위의 과정을 반복해서 모든 원소가 정렬된 모습

정렬된 배열: [8, 11, 18, 25, 26, 31, 32, 45, 83, 84, 88, 94]

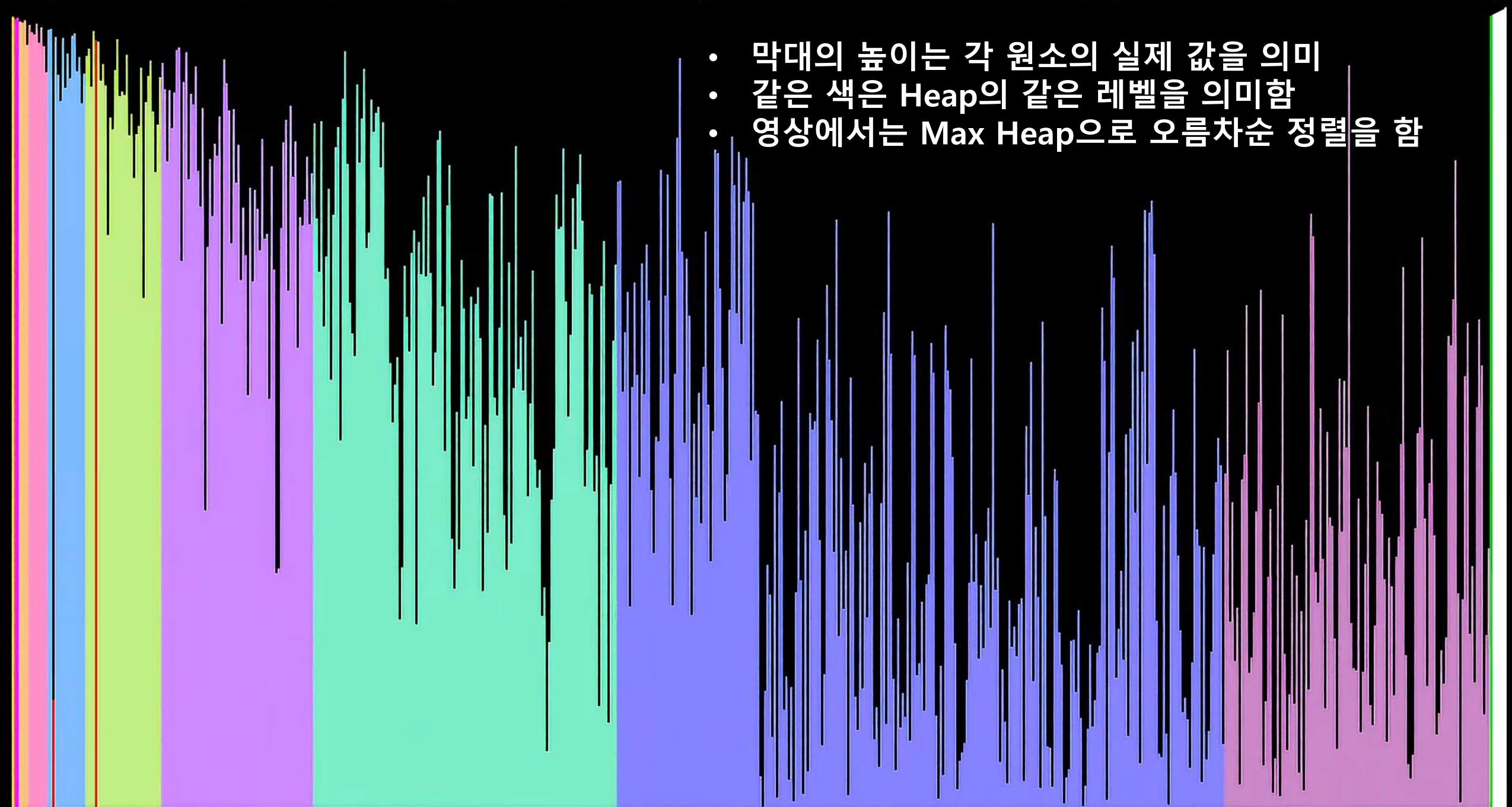


8. 정렬된 배열 반환



힙의 Root에는 항상 최솟값/최댓값이 온다는 사실을 이용

- 막대의 높이는 각 원소의 실제 값을 의미
- 같은 색은 Heap의 같은 레벨을 의미함
- 영상에서는 Max Heap으로 오름차순 정렬을 함



The background is a solid dark blue with abstract geometric shapes. A large, light blue curved shape is on the right side, and a dark blue diagonal band runs from the bottom left towards the top right.

03

DP

Dynamic Programming

재귀함수 (4주차)

03 호출 스택 Call Stack

재귀 함수의 조건

1. Base Case (종료 조건)

재귀 호출을 멈추고 값을 반환하는 탈출 조건임
없으면 무한 루프함

```
if n == 1:  
    return 1 # Base Case
```

2. Recursive Case (재귀 단계)

문제를 더 작은 단위로 분할하여 자기 자신을 호출하는 부분

```
return n * factorial(n - 1)
```

03 호출 스택 Call Stack

재귀 함수의 흐름

factorial(4) 호출 흐름

```
factorial(4) = 4 * factorial(3)  
└─ factorial(3) = 3 * factorial(2)  
    └─ factorial(2) = 2 * factorial(1)  
        └─ factorial(1) = 1 Base Case!
```

Call Stack 반환 과정

factorial(1) [n=1] → 1

factorial(2) [n=2]

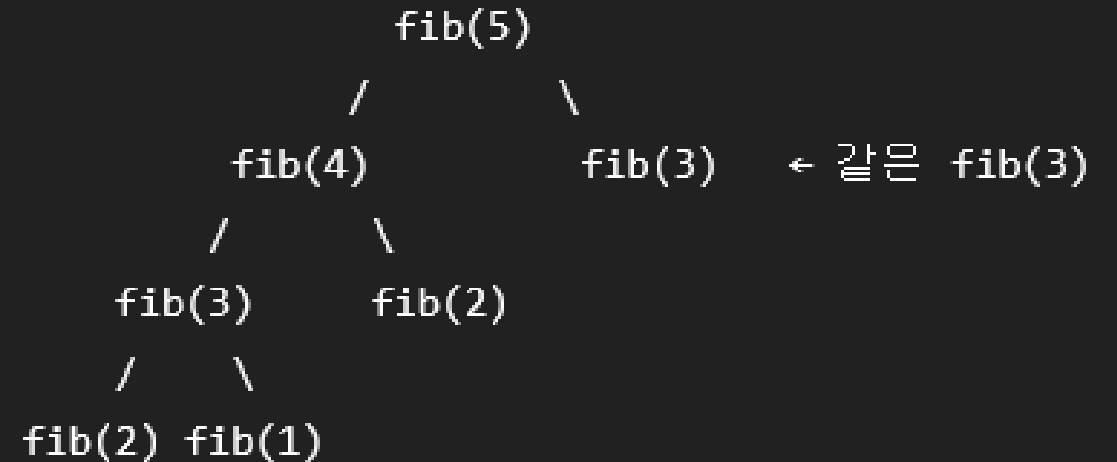
factorial(3) [n=3]

factorial(4) [n=4]

재귀함수의 문제점

- 오른쪽은 피보나치 수열을 계산하는 재귀함수 코드임
- `fib(5)`를 호출하면 `fib` 함수는 약 9번 실행됨
 $fib(5) = fib(4) + fib(3)$
 $fib(4) = fib(3) + fib(2)$
 $fib(3) = fib(2) + fib(1)$
- `fib(50)`을 실행하면 `fib` 함수는 총 **25,172,538,049번 (약 25.1B)** 호출됨
- 단순 재귀함수는 똑같은 계산을 반복하게 됨
- 이 코드는 $O(2^N)$ 의 시간복잡도를 가짐

```
def fib(n):  
    if n <= 2:  
        return 1  
  
    # 단순 재귀 호출  
    return fib(n - 1) + fib(n - 2)
```



Memoization

- 기존의 단순 재귀 코드와 마찬가지로 위에서 아래로 내려가는 Top-down 방식 (fib(5) -> fib(1) 순서로 계산)
- 기존의 코드에 memo를 추가함
- 만약 이전에 계산한 값이 있다면 메모에 저장했다가 꺼내서 사용 (새로 계산하지 않음)
- 시간복잡도를 $O(2^N)$ 에서 $O(N)$ 으로 줄일 수 있음

```
memo = {}
```

```
def fib_topdown(n):
```

```
    if n <= 2:
```

```
        return 1
```

```
    # 이미 계산한 적이 있다면 저장된 값 사용
```

```
    if n in memo:
```

```
        return memo[n]
```

```
    # 처음 계산하는 값이라면 계산 후 저장
```

```
    memo[n] = fib_topdown(n - 1) + fib_topdown(n - 2)
```

```
    return memo[n]
```

Tabulation

- 기존과 달리 가장 처음 단계부터 시작해서 목표 높이까지 올라가는 방식 (fib(1) -> fib(5) 순서로 계산)
- 계산 전에 목표 높이보다 하나 더 많은 0 리스트 선언 (0번 인덱스는 편의상 미사용하는거임)
- DP 테이블에 Base Case (이미 알고 있는 가장 작은 문제) 넣기 (피보나치 수열은 첫번째와 두번째가 각각 1임)
- Base Case를 뺀 나머지는 반복문으로 처리
- 이후 맨 마지막의 목표 높이의 계산 결과 반환
- Call Stack의 스택 오버플로우 위험이 없음 코테 등에서 자주 사용되는 형태

```
def fib_bottomup(n):  
    if n <= 2:  
        return 1  
  
    # DP 테이블 초기화  
    dp = [0] * (n + 1)  
    dp[1], dp[2] = 1, 1  
  
    # 작은 문제부터 차례대로 계산  
    for i in range(3, n + 1):  
        dp[i] = dp[i - 1] + dp[i - 2]  
  
    return dp[n]
```

DP를 적용하기 위한 조건

Overlapping Subproblems

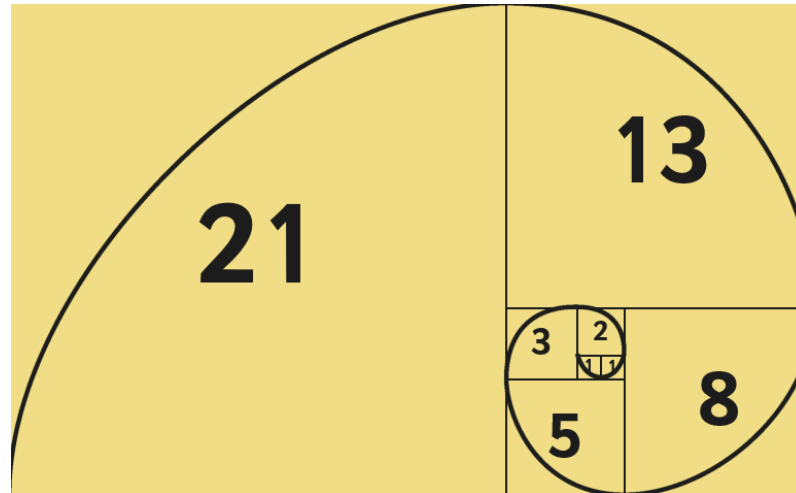
중복 부분 문제

큰 문제를 작은 문제로 나누었을 때,
동일한 작은 문제가 반복해서
등장하는 성질

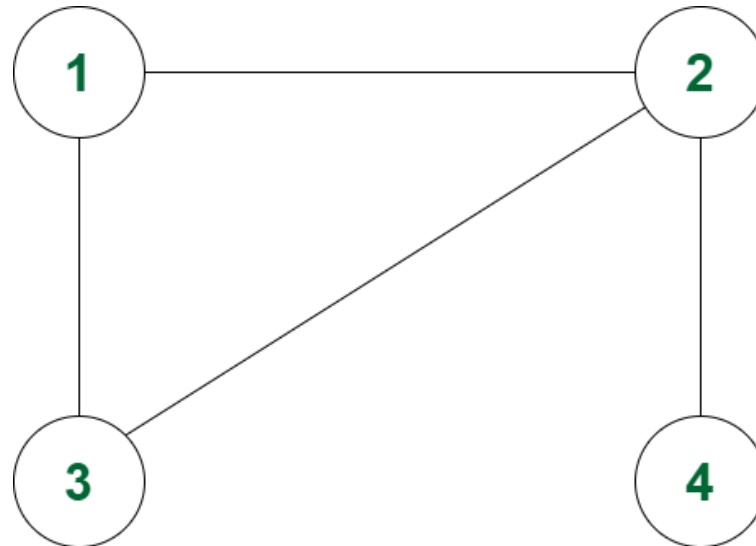
Optimal Substructure

최적 부분 구조

큰 문제의 최적해를 작은
문제들의 최적해를 이용해서
구성할 수 있는 성질



피보나치 수열



최단 거리 문제

3 -> 4 사이의 최단 거리
=
3 -> 2 사이의 최단 거리
+
2 -> 4 사이의 최단 거리

DP의 4단계 절차

STEP 1. DP 배열(State) 정의하기

dp[i]가 정확히 무엇을 의미하는가? 를 문장으로 작성해 보기

예: dp[i]: i번째 계단까지 올라가는 경우의 수

STEP 2. 점화식 (Recurrence) 세우기

dp[i]를 구하기 위해 이전 상태(dp[i-1], dp[i-2])와

어떤 관계가 있는지 관계식 찾기

STEP 3. Base Case (초기값) 설정

가장 작은 문제의 값 직접 지정하기

예: dp[0] = 0, dp[1] = 1

STEP 4. 계산 순서 결정 및 코드 작성

Memoization: N부터 Base Case까지 재귀적으로 내려가며 계산

Tabulation: Base Case부터 N까지 반복문으로 올라가며 계산